

Unit – 7
Function Template and Exception Handling

Object Oriented Programming (CSC166)

BSc CSIT 2nd Semester
Amrit Campus

Prepared By:
Dabbal Singh Mahara

Introduction

- Templates make it possible to use one function or class to handle many different data types.
- Exceptions provide a convenient, uniform way to handle errors that occur within classes.
- These features are combined in a single chapter largely for historical reasons: they became part of C++ at the same time.

Contd..

- In general if the same operation is to be performed with different data types then we need to write the same code(function or classes) for different data types.
- This causes redundant code in the program with only difference in data type.
 - For example, if we need to write functions for max finding (function returns the greater of two values) that support different data types then need to make functions for all different data types.
- But **Templates features** of C++ make it possible to use one function or class to handle many different data types.
- The following program illustrates the need of templates

//program showing the need for template

```
#include<iostream>
```

```
using namespace std;
```

```
int max(int a, int b)
```

```
{  
    int max;  
    if(a>b)  
        max=a;  
    else  
        max=b;  
    return max;  
}
```

```
float max(float a, float b)
```

```
{  
    float max;  
    if(a>b)  
        max=a;  
    else  
        max=b;  
    return max;  
}
```

```
double max(double a, double b)
```

```
{  
    double max;  
    if(a>b)  
        max=a;  
    else  
        max=b;  
    return max;  
}
```

```
}
```

```
char max(char a, char b)
```

```
{  
    int max;  
    if(a>b)  
        max=a;  
    else  
        max=b;  
    return max;  
}
```

```
int main()
```

```
{  
    int x=15, y= 20;  
    cout<<"Greater is:"<<max(x,y)<<endl;  
    float f1=200.34, f2= 2000.234;  
    cout<<"Greater is:"<<max(f1,f2)<<endl;  
    int d1=15.55, d2= 20.45;  
    cout<<"Greater is:"<<max(d1,d2)<<endl;  
    int c1='a', c2= 'A';  
    cout<<"Greater is:"<<max(c1,c2)<<endl;  
    return 0;  
}
```

```
Greater is:20  
Greater is:2000.23  
Greater is:20.45  
Greater is:97
```

What is Template?

- An important feature of c++ which helps to eliminate redundant code by writing a single generic code that takes data type as argument.
- A template allows you to write a single class or function that works with different data types.
- The templates declared for function are called **function templates** and those declared for classes are called **class templates**. They perform appropriate operations depending on the data type of the parameters passed to them.
- The template functions are also called **generic functions** and template class are also called **generic classes** because they support any data type.

Function Template

- In C++, **function templates** are a powerful tool used to write a single, generic function that can work with multiple different data types.
- Instead of manually creating multiple overloaded versions of a function for integers, doubles, or characters, a template allows us to define the function logic once and apply it to any type.
- The main advantage is we only have to write the function code once, making our code cleaner, more reusable, and easier to maintain without needing to manage multiple overloaded functions.

- **Function Template:**

- A function template can be used to operate on different types of data by taking data types as a parameter.
- Template functions are defined by using **the keyword template**.

```
template <class T>
```

```
// or
```

```
template <typename T>
```

```
return_type function_name(parameters)
```

```
{
```

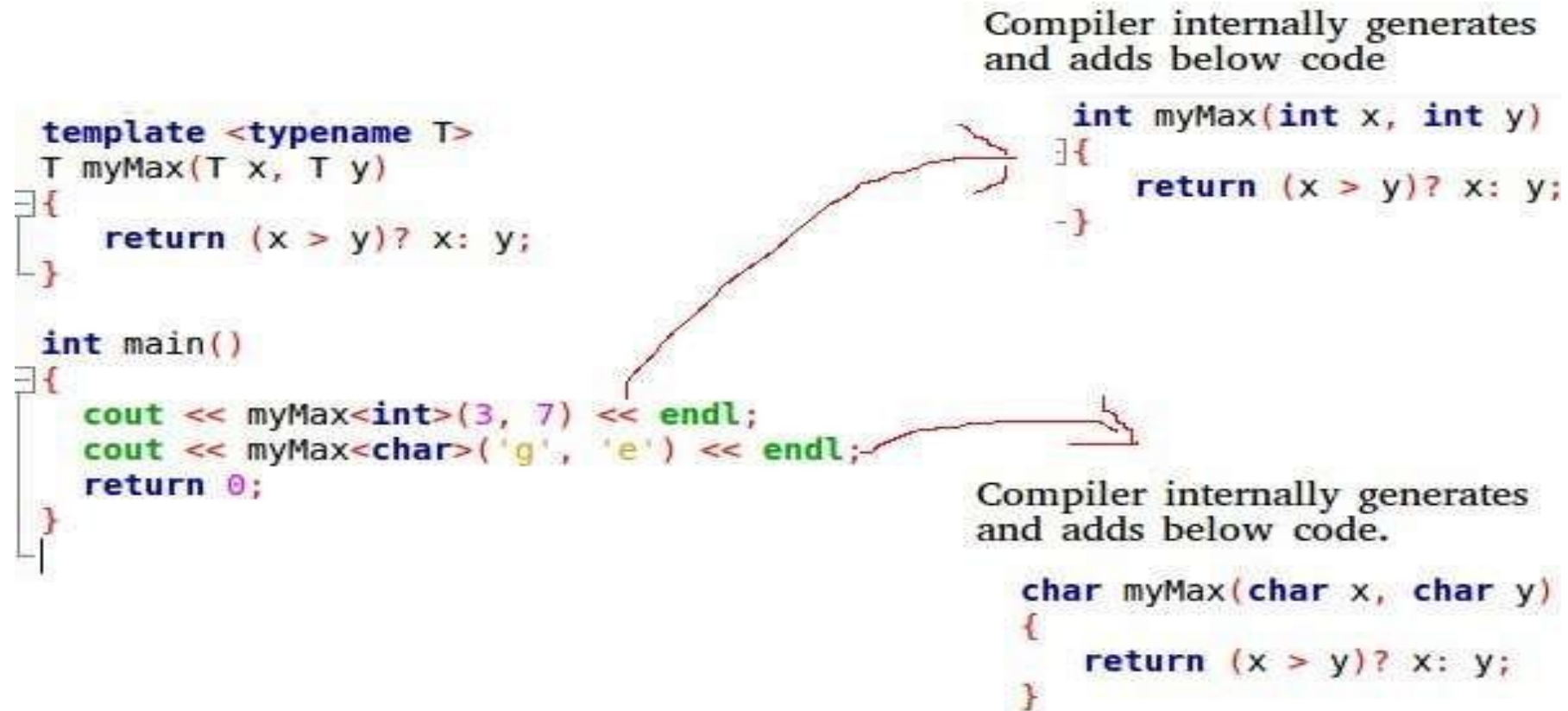
```
    // function body
```

```
}
```

- class **and** typename are equivalent in template declarations.

• How templates work?

- Templates are expanded at compiler time. The idea is simple, source code contains only generic function/class, but compiled code may contain multiple copies of same. function/class.



Contd..

```
//program to find greater using template
#include<iostream>
using namespace std;
template<class T>
T Fmax(T a, T b)
{
    T max;
    if(a>b)
        max=a;
    else
        max=b;
    return max;
}
int main()
{
    int x=15, y= 20;
    cout<<"Greater is:"<<Fmax(x, y)<<endl;
    float f1=200.34, f2= 2000.234;
    cout<<"Greater is:"<<Fmax(f1,f2)<<endl;
    double d1=15.55, d2= 20.45;
    cout<<"Greater is:"<<Fmax(d1,d2)<<endl;
    int c1='a', c2= 'A';
    cout<<"Greater is:"<<Fmax(c1,c2)<<endl;
    return 0;
}
```

```
reater is:20
reater is:2000.23
reater is:20.45
reater is:97
```

- **Homework:** Find minimum of two values by using template function

```
//program to find smaller using template
#include<iostream>
using namespace std;
template<typename T>
T Fmin(T a, T b)
{
    T min;
    if(a<b)
        min=a;
    else
        min=b;
    return min;
}
int main()
{
    int x=15, y= 20;
    cout<<"smaller is:"<<Fmin(x, y)<<endl;
    float f1=200.34, f2= 2000.234;
    cout<<"smaller is:"<<Fmin(f1,f2)<<endl;
    double d1=15.55, d2= 20.45;
    cout<<"smaller is:"<<Fmin(d1,d2)<<endl;
    int c1='a', c2= 'A';
    cout<<"smaller is:"<<Fmin(c1,c2)<<endl;
    return 0;
}
```

```
smaller is:15
smaller is:200.34
smaller is:15.55
smaller is:65
```

- **Function template with multiple arguments:**

- Can be define by specifying more template parameters between the angle brackets in function template declaration.

- For example:

```
template<class T, class U>
T Fmin(T a, U b)
{
    T min;
    if(a<b)
        min=a;
    else
        min=b;
    return min;
}
```

- In this case, our function template Fmin() accepts two parameters of different types and returns value of the same type as the first parameter (T) that is passed.

- For example, after that declaration and definition we could call Fmin() with:

```
y= Fmin(x, z);
```

Contd..

*//program to find minimum of two values of
//different types by using function template*

```
#include<iostream>
using namespace std;
template<class T, class U>
T Fmin(T a, U b)
{
    T min;
    if(a<b)
        min=a;
    else
        min=b;
    return min;
}
int main()
{
    int x=85, y;
    char z= 'a';
    y= Fmin(x, z);
    cout<<"smaller is:"<<y<<endl;
    return 0;
}
```

```
1  #include <iostream>
2  using namespace std;
3
4  template <class T1, class T2>
5  void display(T1 a, T2 b)
6  {
7      cout << "First = " << a << endl;
8      cout << "Second = " << b << endl;
9  }
10
11 int main()
12 {
13     display(100, 25.5);
14     display("Age", 20);
15
16     return 0;
17 }
```

Example: Addition of Different Types

```
1  #include <iostream>
2  using namespace std;
3
4  template <class T1, class T2>
5  auto add(T1 a, T2 b)
6  {
7      return a + b;
8  }
9
10 int main()
11 {
12     cout << add(10, 5.5);
13
14     return 0;
15 }
```

Class Template

- The class template models a generic class which supports similar operation for different data types.
- A **class template** allows a class to work with any data type.
- Instead of creating different classes for integers, floats, or strings, one generic class can be used.

syntax:

```
template<class template_type,.....>
class class_name
{
    private:
        //data member of template type or non template type
    public:
        //function members with template type argument and return type
};
```

Contd..

- The code `template<class template_type,.....>` indicates that a template is being declared.
- The keyword `template` before the `class` indicates that a class template is being declared.
- The keyword `class` inside angle brackets specifies a generic data type(`template_type` in this case).
- The identifier `template_type` is a template parameter that is used as a generic data type within class. There can be more than one template parameters with comma separated list with each template type preceded with keyword `class`.
- The template parameters are replaced by actual data type when the specific version of the class is created.

Contd..

- Once a class template is declared, we create a specific instance of the class using the following syntax:

```
class_name<data_type> object;
```

- Where **data_type** is the type specified as argument to class template. The data type specified here is used by the compiler to replace the template type parameter in the class template definition to create the specific version of the class.
- If int is used as data type then all the occurrence of the template parameter is replaced by int.

Contd..

- If the class template uses **multiple template parameters** then specific instance of class is **created by passing multiple type arguments** as required by the class template declared as:

```
class_name<data_type1, data_type2, .....>object;
```

- The process of generating a class definition from a class template is called **template instantiation**.
- A version of template for a particular template argument is called a **specialization**.

Contd..

- Difference between class template and function template:
 - A class template differ from function template in the way they are instantiated. Calling the function using specific argument type creates a function but to instantiate a class, we must pass the template arguments during object declaration.
 - A class generated from a class template is a perfectly ordinary class. The class template does not add any runtime overhead and it does not reduce amount of code generated but it reduce the redundant code while writing the programs.

Contd..

```
//class template to find larger of two numbers
#include<iostream>
using namespace std;
template<class T>
class mypair
{
    T a,b;
public:
    mypair(T first, T second)
    {
        a=first;
        b= second;
    }
    T getmax()
    {
        T retval;
        retval= a>b?a:b;
        return retval;
    }
};

int main()
{
    mypair <int>o1(100,75);
    cout<<"Larger= "<<o1.getmax()<<endl;
    mypair <double>o2(90.9, 80.8);
    cout<<"Larger= "<<o2.getmax()<<endl;
    return 0;
}
```

```
Larger= 100
Larger= 90.9
```

Templates and inheritance

- Templates and inheritance are two powerful features of C++.
- Templates provide generic programming, allowing one class or function to work with different data types.
- Inheritance provides code reusability, allowing one class (derived class) to inherit properties and functions from another class (base class).
- Combining templates and inheritance allows us to build highly reusable, type-safe hierarchies.
- We can inherit a regular class from a template class, a template class from a regular class, or a template class from another template class.

Case 1: Regular Class Inheriting from a Template Class

- A regular (non-template) class can inherit from a template class by specifying the data type.
- When a non-template class inherits from a class template, derived class specifies the **concrete data type** for the base class during the inheritance declaration.

```
template <class T>
class Base
{
    // members
};
```

```
class Derived : public Base<int>
{
    // members
};
```

Example:

```
#include <iostream>
using namespace std;

template <class T>
class Base
{
protected:
    T value;

public:
    Base(T v)
    {
        value = v;
    }

    void display()
    {
        cout << "Value = " << value << endl;
    }
};
```

```
class Derived : public Base<int>
{
public:
    Derived(int x) : Base<int>(x)
    {
    }

    void square()
    {
        cout << "Square = " << value * value << endl;
    }
};

int main()
{
    Derived d(5);

    d.display();
    d.square();

    return 0;
}
```

OUTPUT:

```
Value = 5
Square = 25
```

Case 2: Template Class Inheriting from a Regular Class

- The base class is a normal class, while the derived class is a template.

```
#include <iostream>
using namespace std;

// Regular base class
class Person
{
protected:
    string name;

public:
    Person(string n)
    {
        name = n;
    }

    void showName()
    {
        cout << "Name : " << name << endl;
    }
};
```

```
// Template derived class
template <class T>
class Student : public Person
{
private:
    T marks;

public:
    Student(string n, T m) : Person(n)
    {
        marks = m;
    }

    void display()
    {
        showName();
        cout << "Marks : " << marks << endl;
    }
};
```

```
int main()
{
    Student<int> s("Ram", 85);
    s.display();

    return 0;
}
```

Case 3: A Template Class Inheriting from Another Template Class

- If we want the derived class to remain generic, we declare the derived class as a template and pass its template parameters directly down to the base class.
- Both the base and derived classes are templates and The same type T is passed to both classes.

Syntax:

```
template <class T>
class Base
{
};
```

```
template <class T>
class Derived : public Base<T>
{
};
```


Example:

```
#include <iostream>
using namespace std;

template <class T>
class Base
{
protected:
    T value;

public:
    Base(T v)
    {
        value = v;
    }

    void show()
    {
        cout << "Value = " << value << endl;
    }
};
```

```
template <class T>
class Derived : public Base<T>
{
public:
    Derived(T x) : Base<T>(x)
    {
    }

    void multiply()
    {
        cout << "Double = " << this->value * 2 << endl;
    }
};
```

```
int main()
{
    Derived<int> d1(10);
    d1.show();
    d1.multiply();
    cout << endl;
    Derived<float> d2(7.5);
    d2.show();
    d2.multiply();
    return 0;
}
```

OUTPUT:

```
Value = 10
Double = 20
```

```
Value = 7.5
Double = 15
```

Case 4: Template Derived Class from a Different Template Base

- The base and derived classes can use different template parameters.

```
#include <iostream>
using namespace std;

template <class T1>
class Base
{
protected:
    T1 x;

public:
    Base(T1 a)
    {
        x = a;
    }
};
```

```
template <class T1, class T2>
class Derived : public Base<T1>
{
    T2 y;

public:
    Derived(T1 a, T2 b) : Base<T1>(a)
    {
        y = b;
    }

    void display()
    {
        cout << "x = " << this->x << endl;
        cout << "y = " << y << endl;
    }
};
```

```
int main()
{
    Derived<int, float> obj(10, 5.5);

    obj.display();

    return 0;
}
```

Exception Handling

- Exceptional Handling (Try, throw and catch)
- Multiple exceptions
- Exceptions with arguments

Exceptions

- An exception is an unwanted or unexpected event that occurs during program execution (at run time) and disrupts the normal flow of a program's instructions.
- Common examples include division by zero, invalid array index access, running out of memory, and invalid type conversions.

Error vs Exception

- **Error:** A serious problem the program usually cannot recover from (e.g., system crash, stack overflow).
- **Exception:** An abnormal condition that a program can anticipate and recover from using exception handling.

What is exception handling?

- Detecting unusual or unexpected conditions of the program at run time and taking preventive measure is called exception handling.
- The purpose of exception handling mechanism is to provide a means to detect and report an exceptional circumstance so that appropriate action can be taken.
- In C++, exception handling attempts to increase reliability by designing system to continue to provide service in spite of the presence of faults.
- It allows the program to continue running or exit gracefully instead of crashing abruptly.

Exception Handling

- Exception handling mechanism in C++ is basically built upon three keywords.

Keyword	Purpose
Try	Defines a block of code that is tested for errors while it executes.
Throw	Used to signal (raise) that an exception has occurred; throws an exception value/object.
Catch	Defines a block of code that executes if a particular exception occurs in the try block.

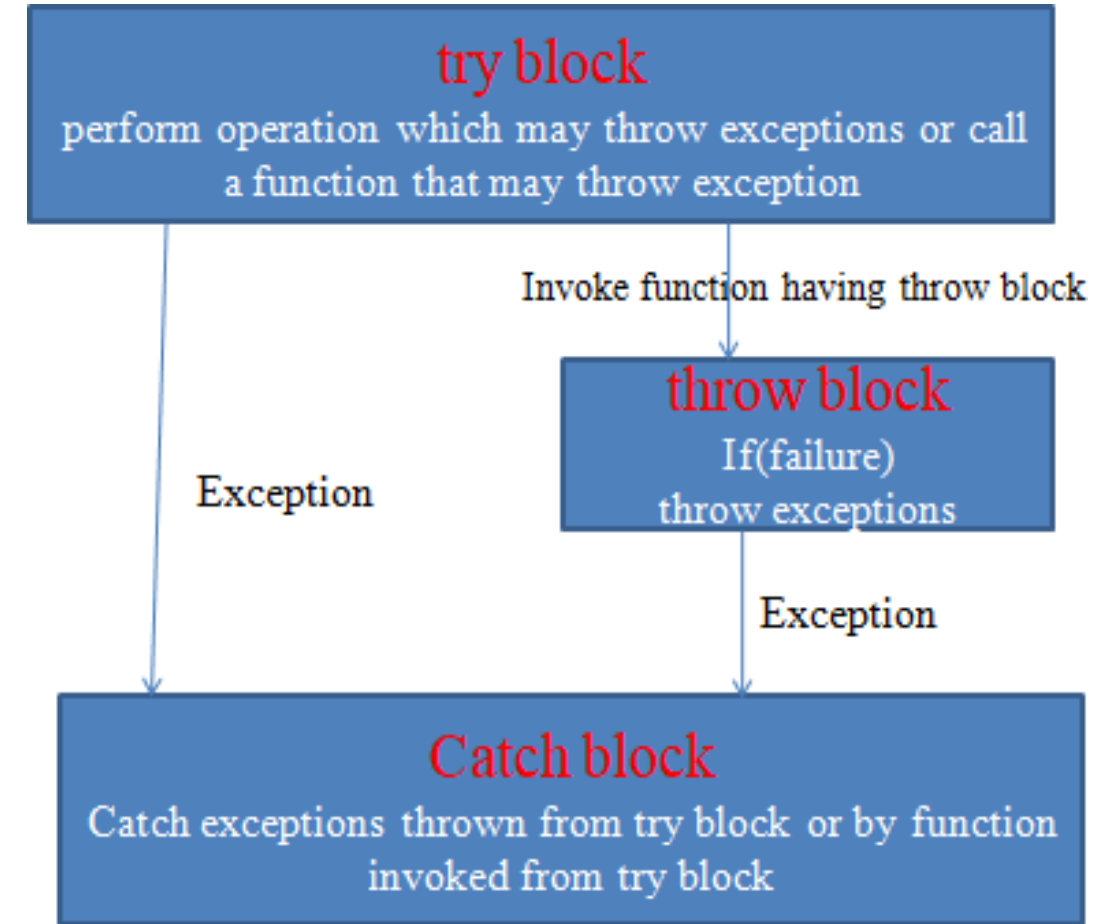


Fig: Exception Handling mechanism in c++

Exception Handling

- Code inside the try block executes normally.
- If a statement inside try triggers a problem, it is thrown using the throw keyword.
- Control immediately leaves the try block (remaining statements in try are skipped).
- The thrown exception is matched against catch blocks in order.
- The first matching catch block executes.
- If no catch block matches, the exception propagates to the calling function and may terminate the program (via terminate()) if never caught.

General Syntax

```
try {  
    // code that may throw an exception  
    if (error_condition)  
        throw exception_value; // can throw int, string, object, etc.  
}  
catch (data_type variable_name) {  
    // code to handle the exception  
}
```


- **try block:**

- The keyword **try** is used to surround a block of statements, which may generate exceptions or a function called from that code generates exception. This block of statements is known as try block.
- The try block is responsible for testing the existence of exceptions.
- When an exception exist then the normal program flow is interrupted and the program control is transferred to the appropriate catch block that matches the type of the object thrown as exception.

- The syntax of try construct is as follows:

```
try
{
    //code that raise exception or

    //call to a function that raise exception

    if (operation fails)
        throw exception;
}
```

- **throw block:**

- When a problem is detected during the execution of code within try block, an exception is raised using keyword **throw**.
- A temporary object of some type is initialized by throw-expression. The type of object is used in matching the catch block.
- The syntax of the throw construct is as follows: **throw [obj];**

- **catch block(exception handler):**
 - The raised exceptions are handled by the catch block.
 - This exception handler is indicated by the keyword **catch**.
 - The catch construct must be used immediately after the try block.
 - The syntax of catch construct is as follows:

```
catch(type exception)
{
    //code to be executed in case of exception
}
```

- A typical exception handling code have the following pattern:

```
function1()
{
    //.....
    if(operation_fails)
        throw obj1;
}
function2()
{
    try
    {
        //.....
        function1();

        if (operation fails)
            throw obj2;
    }
    catch(type1 obj1)
    {
        //handle appropriate actions
    }
    catch(type2 obj2)
    {
        //handle appropriate actions
    }
    //.....
}
```

Contd..

- The code that handles exception should perform the following task:
 - Hit the exception (detect the problem causing exception)
 - Throw the exception(inform that an error has occurred)
 - Catch the exception (receive the error information)
 - Handle the exception(take appropriate actions)
- When an exception is raised following sequence of steps are performed
 - First, the program searches for a matching handler.
 - if any matching handler is found, the program control is transferred to the handler
 - if handler is not found the program will call the function terminate().
- If no exception are thrown, the program executes in normal way.

Example1:

```
//try block throwing exception
#include<iostream>
using namespace std;
int main()
{
    int a;
    int b;
    cout<<"Enter the value of a and b:"<<endl;
    cin>>a>>b;
    try
    {
        if(b==0)
            throw b;
        else
            cout<<"Result="<<(float)a/b;
    }
    catch(int i)
    {
        cout<<"Divide by zero exception:b="<<i;
    }
    return 0;
}
```

First Execution:

```
Enter the value of a and b:
50
5
Result=10
-----
```

Second Execution:

```
Enter the value of a and b:
5
0
Divide by zero exception:b=0
-----
```

- **Example2:**

Contd..

```
//Function invoked by try block thowing exception
#include<iostream>
using namespace std;
void divide(int a, int b)
{
    if(b==0)
        throw b;
    else
        cout<<"Result="<<(float)a/b;
}
int main()
{
    int a;
    int b;
    cout<<"Enter the value of a and b:"<<endl;
    cin>>a>>b;
    try
    {
        divide(a,b);
    }
    catch(int i)
    {
        cout<<"Divide by zero exception:b="<<i;
    }
    return 0;
}
```

First Execution:

```
Enter the value of a and b:
50
5
Result=10
-----
```

Second Execution:

```
Enter the value of a and b:
5
0
Divide by zero exception:b=0
-----
```


Multiple Exceptions:

- In a program there is a chance of errors occurring more than once at runtime.
- In such a case, C++ permits to design functions to throw as many exceptions as needed.

Example:

// program with multiple exceptions and multiple handlers

```
#include<iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int size;
```

```
    cout<<"Enter size:"<<endl;
```

```
    cin>>size;
```

```
    try
```

```
    {
```

```
        char *mystring;
```

```
        if (size<=0)
```

```
            throw 's';
```

```
        mystring = new char [size];
```

```
        for(int n=0;n<=100;n++)
```

```
        {
```

```
            cout<<n<<endl;
```

```
            if(n>size-1)
```

```
                throw n;
```

```
        }
```

```
    }
```

```
    catch(int i)
```

```
    {
```

```
        cout<<"Exception!";
```

```
        cout<<"Index"<<i<<"is out of range"<<endl;
```

```
    }
```

```
    catch(char c)
```

```
    {
```

```
        cout<<"Exception!";
```

```
        cout<<"size must be non-zero positive number";
```

```
    }
```

```
    return 0;
```

```
}
```

Enter size:

-10

Exception!size must be non-zero positive number

Enter size:

20

0

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

Exception!Index20is out of range

Exception with arguments:

- throw objects(exception) with information about the cause of exception.
- The exception with argument helps the programmer to know what bad value actually caused the exception.

Contd..

- To throw exception with arguments define the exception class with members.
i.e., Declare exception classes to throw object with information of the cause of the error.
- The catch block mention type and object to catch and get the object thrown from the exception as:

```
try
{
    //throw object
}
catch(class_name obj)
{
    // handle the exception as according to the value of obj
    //with the help of the exception object in parameter, the
    //description information of the cause of the exception
    //can be provided to the exception handler
}
```

Example:

```
//Exception with arguments |
#include<iostream>
#include<cstring>
#include<cmath>
using namespace std;
class number
{
private:
    double num;
public:
    class Neg
    {
    public:
        double value;
        char description[20];
        Neg()
        {
            value=0;
            description[0]='\0';
        }
        Neg(double v, char *desc)
        {
            value=v;
            strcpy(description, desc);
        }
    };
    void readnum()
    {
        cout<<"Enter number:";
        cin>>num;
    }
};
```

```
double sqrt()
{
    if(num<0)
        throw Neg(num,"Negative Number");
    else
        return (sqrt(num));
};
int main()
{
    number n1;
    double r;
    n1.readnum();
    try
    {
        r=n1.sqrt();
        cout<<"square root:"<<r<<endl;
        cout<<"success!";
    }
    catch(number::Neg n)
    {
        cout<<"Exception!"<<endl;
        cout<<"Trying to find square root of "<<n.value<<endl;
        cout<<"Error description:"<<n.description;
    }
    return 0;
}
```

```
Enter number:25
square root:5
success!
```

```
Enter number:-25
Exception!
Trying to find square root of -25
Error description:Negative Number
-----
```

Why Exception Handling?

- Following are main advantages of exception handling over traditional error handling.
 - Separation of Error Handling code from Normal Code
 - Methods can handle any exceptions they choose
 - Grouping of Error Types

Contd..

- **Separation of Error Handling code from Normal Code:**
 - In traditional error handling codes, there are always if else conditions to handle errors. These conditions and the code to handle errors get mixed up with the normal flow. This makes the code less readable and maintainable. With try catch blocks, the code for error handling becomes separate from the normal flow.

Contd..

- **Methods can handle any exceptions they choose:**
 - A function can throw many exceptions, but may choose to handle some of them. The other exceptions which are thrown, but not caught can be handled by caller. If the caller chooses not to catch them, then the exceptions are handled by caller of the caller.
 - In C++, a function can specify the exceptions that it throws using the `throw` keyword. The caller of this function must handle the exception in some way (either by specifying it again or catching it)

Contd..

- **Grouping of Error Types:**
 - In C++, both basic types and objects can be thrown as exception. We can create a hierarchy of exception objects, group exceptions in namespaces or classes, categorize them according to types.

Homework

- What is template? State and explain the importance of template.
- Explain the function template with example.
- What is function template? Differentiate it with class template.
- Write a program to find sum of two number with different data types by using template functions.
- Explain the exception handling with example.
- Discuss different keywords used in exception handling.
- Why do we need exceptions? Explain “exceptions with arguments” with suitable program.
- What are the advantages of exception handling over conventional error handling.